



Princess Sumaya جامعة
University الأميرة سميرة
for Technology للتكنولوجيا

Princess Sumaya University for Technology

King Abdullah II School for Electrical Engineering

Computer Engineering Department

Networks Security

25420

Hala AL Zaben

Introduction:

This report documents the process of extracting the WEP key, protocol type, and hostname from a capture file named Puzzle.cap, which contains only three encrypted packets. Traditional WEP cracking techniques failed due to an insufficient number of Initialization Vectors (IVs). To overcome this, we used a dictionary attack with the darkc0de.lst wordlist and manually analyzed the packets using Wireshark and a custom Python script. This assignment highlights the weaknesses of WEP encryption, demonstrating how it can be broken even with limited captured data

Objectives:

- Crack a WEP key from a .cap file that includes only three encrypted packets.
- Identify the correct WEP key using a custom wordlist.
- Analyze the decrypted packets to detect the network protocols involved.
- Extract the hostname from the decrypted communication.
- Document all tools, steps, and findings clearly in the report.

Tools used:

- Python 3 used (**Ubuntu**)
- Wireshark
- darkc0de.lst (wordlist)

- **Step 1:**

First, the darkc0de.lst wordlist must be downloaded in order to perform a dictionary attack. This can be done using the following command

```
ubuntu@ubuntu2004:~/Downloads$ wget https://github.com/danielmiessler/SecLists/raw/master/Passwords/darkc0de.txt
--2026-01-05 02:11:55-- https://github.com/danielmiessler/SecLists/raw/master/Passwords/darkc0de.txt
Resolving github.com (github.com)... 140.82.121.3
Connecting to github.com (github.com)|140.82.121.3|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/danielmiessler/SecLists/master/Passwords/darkc0de.txt [following]
--2026-01-05 02:11:55-- https://raw.githubusercontent.com/danielmiessler/SecLists/master/Passwords/darkc0de.txt
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.111.133, 185.199.110.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.111.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 15069474 (14M) [text/plain]
Saving to: 'darkc0de.txt.1'

darkc0de.txt.1          100%[=====>] 14.37M  4.33MB/s   in 3.5s

2026-01-05 02:12:00 (4.13 MB/s) - 'darkc0de.txt.1' saved [15069474/15069474]
```

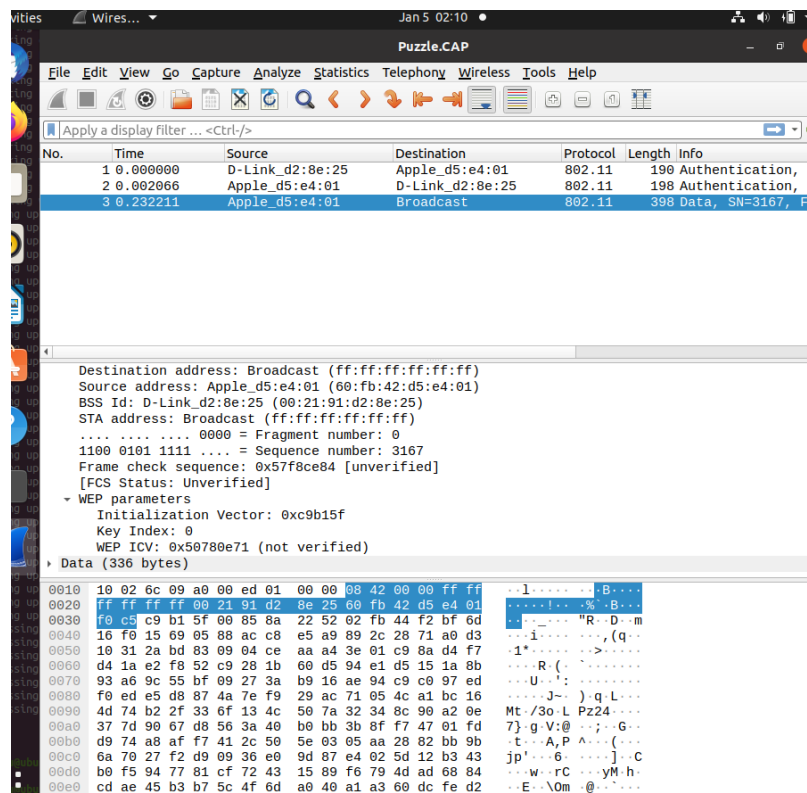
- **Step 2:**

Next, we used the .cap file

This file contained three encrypted packets:

- **A Challenge Response**
- **An Encrypted Challenge,**
- **A Data Packet.**

Then went to the packet that included the Encrypted Challenge and copied the encrypted data (ciphertext)

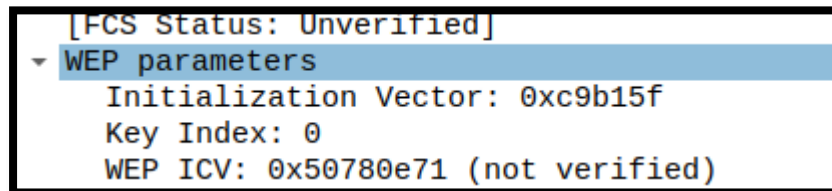


The challenge:

858a225202fb44f2bf6d16f015690588acc8e5a9892c2871a0d310312abd830904cea
aa43e01c98ad4f7d41ae2f852c9281b60d594e1d5151a8b93a69c55bf09273ab916ae
94c9c097edf0ede5d8874a7ef929ac71054ca1bc164d74b22f336f134c507a32348c9
0a20e377d9067d8563a40b0bb3b8ff74701fdd974a8aff7412c505e0305aa2882bb9b
6a7027f2d90936e09d87e4025d12b343b0f5947781cf72431589f6794dad6884cdae4
5b3b75c4f6da040a1a360dcfed2b1dcbbfb238d1acf4af9783951bdd950e79b26d958
ba8a741dae14c92f2c1e95ea42a2c52b1141503497f759f0eb440335dc5901fc522f80
58b6abf31430d32443a544f98b2704d2676ae416c0a8e11aa329ce202c2ecc73c91ee
2b291109db6202bdb00a5cd7a823685268ccc9761d757258866cc5094974756cd1b
46e1e58057d7f2ffbc7c4575f5d7703b2b02c5b690d894f14067

- **Step 3:**

Also noted the IV (Initialization Vector), which we found under the IEEE 802.11 layer in Wireshark.



The IV=0xc9b15f

The code implements a brute-force approach to recover a WEP key using the RC4 algorithm. First, it defines the rc4() function, which performs both the Key Scheduling Algorithm (KSA) and the Pseudo-Random Generation Algorithm (PRGA) to decrypt data using a given key. In the main part of the script, the Initialization Vector (IV) and encrypted challenge are provided. The script then opens a wordlist file (drarkc0de.txt.1) and iterates through each line, checking for keys that are exactly 5 characters long (the typical size for 40-bit WEP). For each valid key, it concatenates it with the IV to form the RC4 input key, then decrypts the challenge using the rc4() function. If the decrypted output contains the known WEP plaintext pattern (\xAA\xAA\x03\x00\x00\x00), it prints the matching key and stops the loop

```
GNU nano 4.8 crack_wep.py
#!/usr/bin/env python3

def rc4(data, key):
    S = list(range(256))
    j = 0
    for i in range(256):
        j = (j + S[i] + key[i % len(key)]) % 256
        S[i], S[j] = S[j], S[i]
    i = j = 0
    result = []
    for byte in data:
        i = (i + 1) % 256
        j = (j + S[i]) % 256
        S[i], S[j] = S[j], S[i]
        k = S[(S[i] + S[j]) % 256]
        result.append(byte ^ k)
    return bytes(result)

iv_hex = "c9b15f"
challenge_hex = "858a225202fb44f2b6d16f015690588acc8e5a9892c2871a0d310312abd830904ceaaa43e01c98ad4f7d41ae2f852c9281b60d594e1d5151a8b93a69c55bf09273ab916ae94c9c097edf0ede5d8874a7ef92"

iv_bytes = bytes.fromhex(iv_hex)
challenge_bytes = bytes.fromhex(challenge_hex)
known_plaintext = b'\xaa\xaa\x03\x00\x00'

print("Starting WEP crack...")
print(f"IV: {iv_hex}")
print("Testing 5-character keys...")

with open('dark0de.txt.1', 'r', errors='ignore') as f:
    for line_num, line in enumerate(f, 1):
        key_candidate = line.strip()

        if len(key_candidate) == 5:
            rc4_key = iv_bytes + key_candidate.encode('latin-1')
            decrypted = rc4(challenge_bytes, rc4_key)

            if known_plaintext in decrypted:
                print(f"\nKEY FOUND: '{key_candidate}'")
                print(f"Hex: {key_candidate.encode().hex()}")
                break

        if line_num % 100000 == 0:
            print(f"Processed {line_num} lines...", end='\r')
```

- **Step 4:**

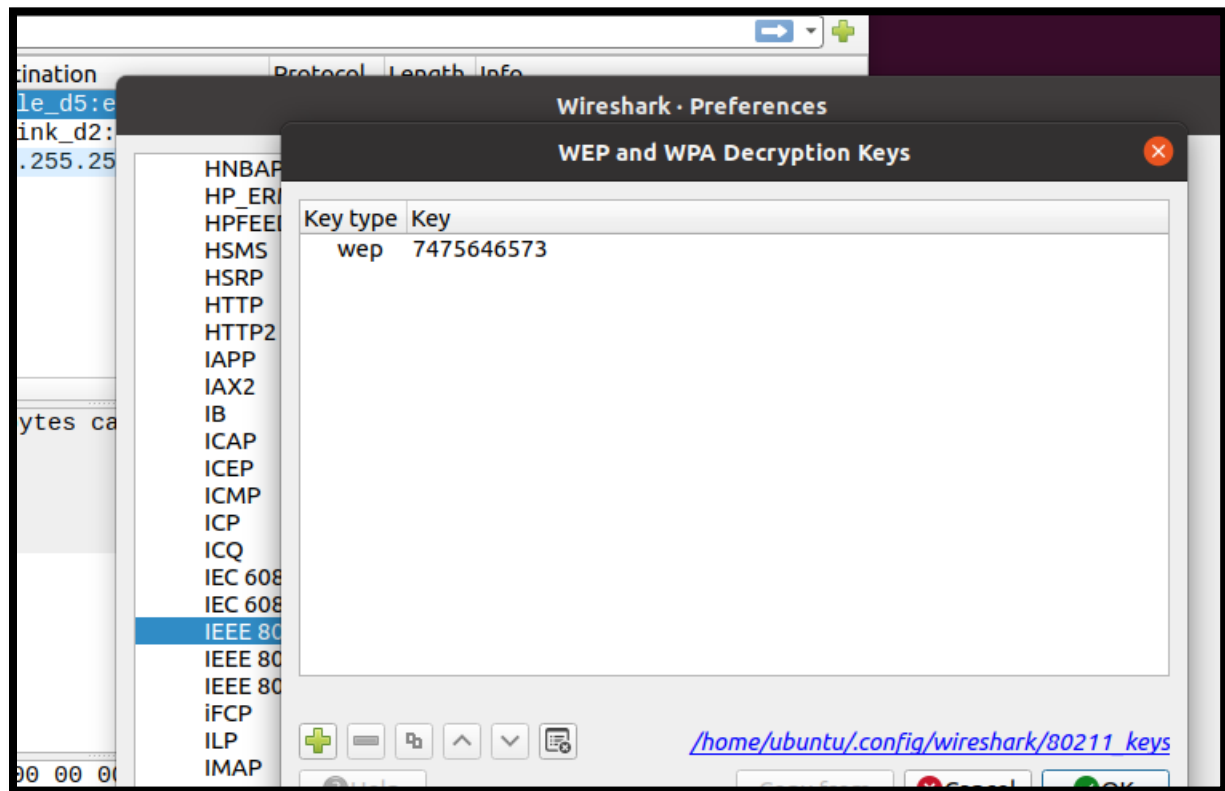
After running the script, I was able to obtain the correct WEP key

```
ubuntu@ubuntu2004:~/Downloads$ python3 crack_wep.py
Starting WEP crack...
IV: c9b15f
Testing 5-character keys...
Processed 1300000 lines...
KEY FOUND: 'tudes'
Hex: 7475646573
```

- **Step 5:**

Then I went back to Wireshark. We opened *Edit > Preferences > Protocols > IEEE 802.11*, then clicked *Edit* under the decryption keys section. There I added the correct key after convert it to a hexadecimal (the converting process was in Ubuntu)

The key in hexadecimal in Wireshark :



After adding the key, the data in Wireshark became readable. We were able to see several layers in the packet like:

- Logical Link Control (LLC)
- Internet Protocol version 4 (IPv4)
- User Datagram Protocol (UDP)
- Dynamic Host Configuration Protocol (DHCP)

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	D-Link_d2:8e:25	Apple_d5:e4:01	802.11	190	Authentication, SN=3162, FN=0, Flags=.....C
2	0.002066	Apple_d5:e4:01	D-Link_d2:8e:25	802.11	198	Authentication, SN=1208, FN=0, Flags=.p.....C
3	0.232211	0.0.0.0	255.255.255.255	DHCP	398	DHCP Request - Transaction ID 0xa7043488

Frame 3: 398 bytes on wire (3184 bits), 398 bytes captured (3184 bits)

- Radiotap Header v0, Length 26
- 802.11 radio information
- IEEE 802.11 Data, Flags: .p....F.C
- Logical-Link Control
- Internet Protocol Version 4, Src: 0.0.0.0, Dst: 255.255.255.255
- User Datagram Protocol, Src Port: 68, Dst Port: 67
- Dynamic Host Configuration Protocol (Request)

Step 6:

Then I went into the DHCP packet, scrolled down to the options section, and found the hostname listed there. That confirmed that the key worked and the decryption was successful

Dynamic Host Configuration Protocol (Request)
Message type: Boot Request (1)
Hardware type: Ethernet (0x01)
Hardware address length: 6
Hops: 0
Transaction ID: 0xa7043488
Seconds elapsed: 0
Option: (12) Host Name
Length: 13
Host Name: viveks-iphone
Option: (255) End

Concloution :

This assignment taught us that even when only a few packets are captured, WEP can still be broken using custom methods like dictionary attacks. By analyzing encrypted packets carefully and using scripting tools, we were able to get the WEP key, find the protocol types, and extract the hostname from the decrypted traffic. This shows just how unsafe WEP really is and why more secure encryption methods should always be used in wireless networks.